Apellidos y nombre: ..... Grupo: ..... DNI: .....

Examen final – primera parte 1 (temas 2, 3, 4, 5, 6, 7, 8)

Duración del examen: **2:00 horas**. La solución de cada ejercicio se tiene que escribir en el **espacio reservado** para ello en el **propio enunciado**. No podéis utilizar calculadora, móvil, apuntes, etc. La solución del examen se publicará en Atenea mañana.

Pregunta 1 (1 punto)

a) (0,5 puntos) Cada fila de la tabla tiene 4 columnas con: el vector X de 8 bits, X expresado en hexadecimal, el valor en decimal, Xu, que representa X interpretado como un número natural codificado en binario, y el valor en decimal, Xs, que representa X interpretado como un número entero codificado en binario en Ca2. Completa todas las casillas vacías.

X (binario)	X (hexadecimal)	Xu	Xs
01011100	0x5C	92	92
01111110	0x7E	126	126
00010011	0x13	19	19
11110010	0xF2	242	-14

-0,25 por error.

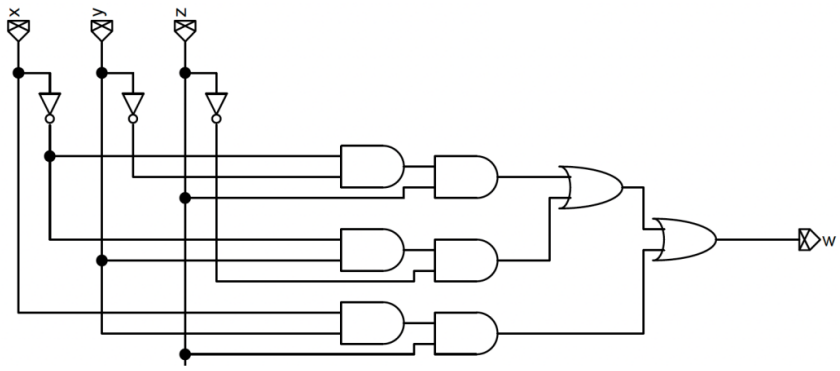
b) (0,5 puntos) Fórmulas para rango y codificación binaria natural y Ca2 para n bits.

	Binario Natural	Ca2
Codificación	$Xu = \sum_{i=0..n-1} x_i * 2^i$	$Xs = -2^{n-1} * x_{n-1} + \sum_{i=0..n-2} x_i * 2^i$
Rango	$0 <= Xu <= 2^n-1$	$-2^{n-1} <= Xs <= 2^{n-1}-1$

-0,25 por columna con error.

Pregunta 2 (1 punto) Dada la siguiente tabla de verdad, sintetiza el circuito lógico combinacional que define la función lógica w como suma de minterms utilizando sólo puertas NOT, AND-2 y OR-2.

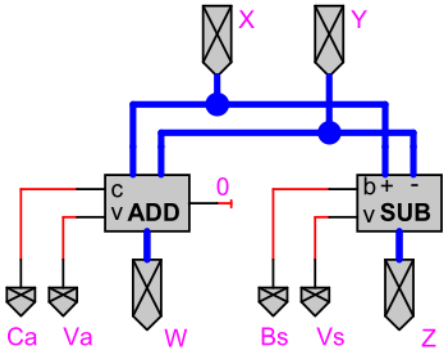
X	Y	Z	W
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



-0,25 por error

Apellidos y nombre: ..... Grupo: ..... DNI: .....

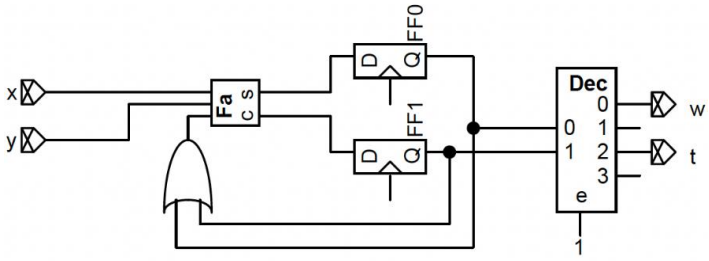
Pregunta 3 (1.5 puntos) Dado el esquema del circuito combinacional a bloques, donde los buses son de 8 bits, completad las columnas que faltan, para cada una de las tres filas (casos concretos de valores de las entradas). Los vectores de bits X, Y, W y Z se indican en hexadecimal. Las interpretaciones de los vectores de bits de las salidas W y Z (con subíndices u y s según se consideren números naturales o enteros) se deben expresar en decimal sin y con signo respectivamente).

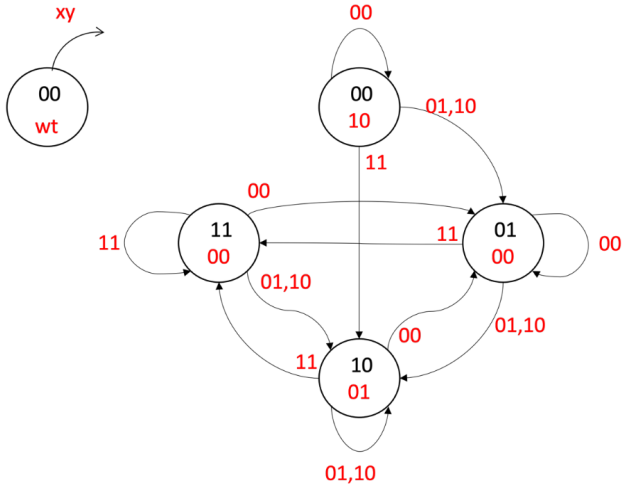


	X	Y	Ca	Va	W	Wu	Ws	Bs	Vs	Z	Zu	Zs
a	1E	D6	0	0	F4	244	-12	1	0	48	72	72
b	FD	A9	1	0	A6	166	-90	0	0	54	84	84
c	53	AB	0	0	FE	254	-2	1	1	A8	168	-88

-0,50 por fila/columna con error. Se toma el mínimo valor filas/columnas incorrectas.

Pregunta 4 (1,5 puntos) Dado el siguiente CLS:
a) (1 punto) Completar el grafo de estados del siguiente circuito secuencial. No olvidéis la leyenda.





-0,25 por estado incorrecto.

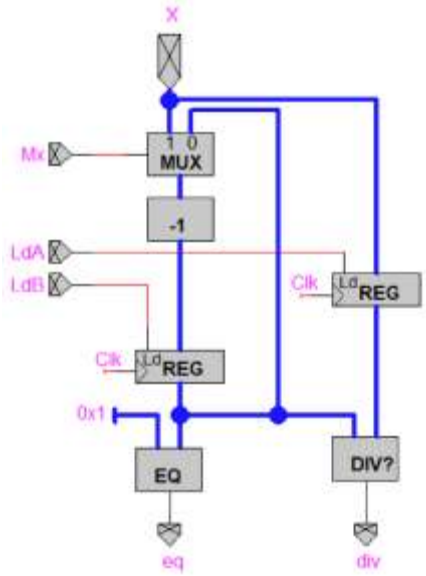
b) (0.5 puntos) ¿Cuál sería el camino crítico y el tiempo de propagación del CLS? Suponiendo los siguientes tiempos de propagación: TP (**FA**) = 120 u.t. TP (**OR-2**) = 20 u.t. TP (**Biestable**) = 100 u.t. TP (**MUX**) = 100 u.t. TP (**Dec**) = 40 u.t. TP (**ROM**) = 50 u.t. Las **entradas "x"** y **"y"** tardan 175 u.t. en estabilizarse. La **salida w** debe estar estable 30 u.t. antes del final de ciclo y la salida t 75 u.t.

Camino Crítico	Tiempo Propagación
x/y -> FA -> FF0/1	175+120 = 295ut

-0,25 por cada valor incorrecto.

Apellidos y nombre: ..... Grupo: ..... DNI: .....

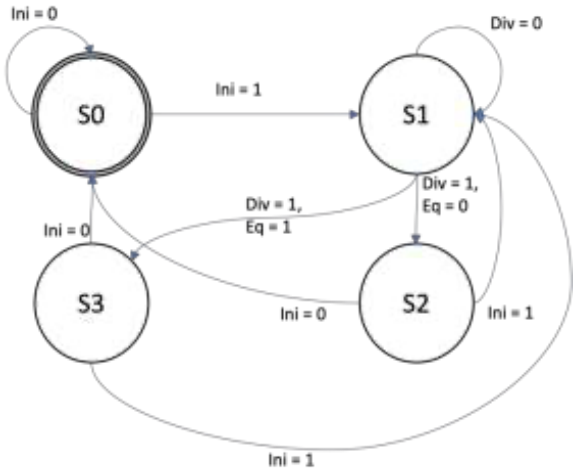
Pregunta 5 (2 puntos) Queremos implementar un Procesador de Propósito Específico (PPE) que calcula si un número es primo o no (número divisible sólo por sí mismo y por 1). Para ello disponemos de la Unidad de Proceso de 16 bits de la figura. En dicha figura, el componente **"DIV"** tiene como entrada dos números de 16 bits y retorna si son divisibles entre sí (1 si lo son, 0 si no lo son). El circuito funciona de la siguiente forma. Tiene dos entradas: un bus de entrada de 16 bits y una señal de control **ini**. También tiene dos señales de salida: **fi** y **primo**. Cuando **ini** toma valor 1, en X tenemos el número que queremos comprobar si es o no primo. Pasados los ciclos que sean necesarios, la señal **fi** valdrá 1 y **primo** valdrá 1 si el número es primo y 0 si no lo es. Mientras **fi** valga 0, el valor de primo no importa. Ignoraremos la señal de **ini** durante los cálculos e intentaremos poder empezar un nuevo cálculo lo antes posible. El valor recibido per el bus X siempre será mayor que 1.



1) (1,5 puntos) Diseña la Unidad de Control completando el siguiente grafo. Especifica las palabras de condición y control del PPE.

Palabra de Control: Mx LdA LdB

Palabra de Condición: Div Eq



	Mx	LdA	LdB	fi	primo
S0	1	1	1	0	x
S1	0	0	1	0	x
S2	1	1	1	1	0
S3	1	1	1	1	1

-0,25 palabra de condición, -0,25 palabra de control, -0,25 por estado incorrecto

2) (0,5 puntos) Si implementamos la unidad de control con 1 ROM y 1 Multiplexor de buses

¿Cuántas palabras tendrá la ROM?:	2^2 = 4
¿Cuántos bit por palabra?:	2 x 2^3 + 5 = 21
¿Cuántos bits tendrá el registro de estado?:	2
¿Cuántos bits tendrá el selector del multiplexor de buses?:	3

-0,25 por valor incorrecto

Pregunta 6 (1 punto) Indicad el valor que debe tener cada uno de los bits de la palabra de control de la UPG (sin subsistema de I/O ni memoria) para que realice, durante un ciclo, la acción concreta especificada mediante el mnemotécnico. Indicad con X las casillas cuyo valor no importe para la ejecución de la instrucción. En caso de que no se pueda realizar la acción, tachad toda la línea de señales.

Mnemotécnico	@A	@B	Rb/N	OP	F	In/ALU	@D	WrD	N (Hexa)
SUBI R1, R1, -2	001	xxx	0	00	101	0	001	1	0xFFFE
OUT R2 // IN R1	010	xxx	x	xx	xxx	1	001	1	0xXXXX
SHLI R5, R7, 1	111	xxx	0	00	111	0	101	1	0x0001
ADD R7, R6, R5	110	101	1	00	100	1	111	1	0xXXXX

-0,25 por fila/columna incorrecto/a. Se toma el mínimo valor para filas/columnas incorrectas.

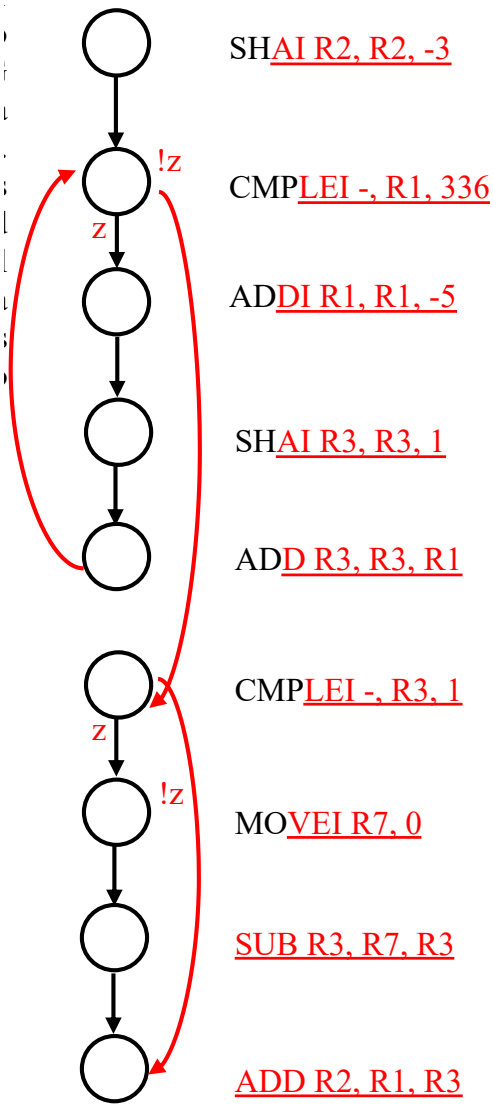
Apellidos y nombre: ..... Grupo: ..... DNI: .....

Pregunta 7 (2 puntos) Dado el siguiente código en C (el código no tiene que hacer algo útil; los datos deben interpretarse como valores enteros)

```
R2 = R2 / 8;
while ( R1 > 336 ) {
  R1 = R1 – 5;
  R3 = R3 * 2 + R1;
}
if ( R3 > 1 ) { R3 = -R3; }
R2 = R1 + R3;
```

Apartado a), -0,25 por estado incorrecto. Apartado b) -0,25 por instrucción incorrecta.

a) (1 punto) Completad el fragmento de grafo de estados de la UC de propósito específico para que junto con la UPG formen un procesador que realice la funcionalidad descrita en el código. Indicad los arcos que faltan, las etiquetas de los arcos (z, !z, o nada) y completad las casillas de cada palabra de control que se especifica con mnemotécnicos a la derecha de cada nodo del grafo. Si es necesario, podéis utilizar R7 como registro temporal.



b) (1 punto) Completad el fragmento de programa en lenguaje ensamblador SISA para que el procesador formado por la unidad de control de propósito general (UCG) junto con la UPG realice la funcionalidad descrita en el código. Rellenad la parte que falta. No se pueden utilizar etiquetas para los saltos. R7 se ha utilizado como registro temporal

```
MOVI   R7,  -3
SHA    R2, R2, R7
MOVI   R7,  80
MOVHI  R7,  1
CMPLT  R7,  R7,  R1
BZ     R7,  5
MOVI   R7,  5
SUB    R1,  R1,  R7
ADD    R3,  R3,  R3
ADD    R3,  R3,  R1
BNZ    R7,  -9
MOVI   R7,  1
CMPLE  R7, R3, R7
BNZ    R7,  2
NOT    R7, R3
ADDI   R3, R7, 1
ADD    R2, R1, R3
```